

Supplementary Material

JAEHYUN LEE, University of Wisconsin-Madison, USA

EFTYCHIOS SIFAKIS, University of Wisconsin-Madison, USA and NVIDIA, USA

ACM Reference Format:

JaeHyun Lee and Eftychios Sifakis. 2026. Supplementary Material. In *SIG-GRAPH Asia 2026 Conference Papers (SA Conference Papers '26)*, December 01–04, 2026, Kuala Lumpur, Malaysia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3829340.3842319>

1 Pseudocode

Algorithm 1: RHS Homogenization. Pre-processes the right-hand side $\mathbf{f}$ to absorb non-homogeneous boundary terms. It subtracts the operator action on fixed Dirichlet values $\mathbf{g}$ and adds the affine forcing terms arising from ghost-point elimination at natural boundaries, effectively converting the problem into a homogeneous linear system $\mathbf{L}\mathbf{u} = \hat{\mathbf{f}}$.

Algorithm 2: Diagonal Extraction. Computes the diagonal elements $\mathbf{D}$ of the operator matrix $\mathbf{L}$, which are required for stationary smoothing (e.g., Jacobi or Gauss-Seidel). The routine aggregates the central weight of the uniform stencil with the specific diagonal shifts introduced by the Robin/Neumann boundary corrections defined in the main text.

Algorithm 3: Convolutional V-Cycle. Implements a geometric multi-grid V-cycle entirely via tensor primitives. The hierarchy mimics a U-Net architecture: restriction is performed via stride-2 convolution (pooling), prolongation via stride-2 transposed convolution (upsampling), and smoothing via weighted Jacobi iterations.

Algorithm 4: Adjoint Operator Action. Evaluates the transposed operator action $\mathbf{v} \mapsto \mathbf{L}^T \mathbf{v}$ for reverse-mode differentiation. By exploiting the property that the adjoint of a convolution uses the spatially flipped kernel, this routine propagates sensitivities backwards through the boundary masking and coefficient multiplication steps without requiring manual derivative kernels.

Algorithm 5: Hybrid Gradient Computation. Computes the gradients of the loss function $\mathcal{L}$ with respect to the right-hand side $\mathbf{f}$ and physical parameters θ . It employs a hybrid strategy: first, it solves the adjoint linear system $\mathbf{L}^T \boldsymbol{\lambda} = \partial \mathcal{L} / \partial \mathbf{u}$ using the transposed operator action (Algorithm 4) to obtain the adjoint vector $\boldsymbol{\lambda}$. Then, it leverages automatic differentiation for the vector-Jacobian product (VJP) by backpropagating $\boldsymbol{\lambda}$ through a single forward evaluation of the operator $\mathbf{L}(\theta)\mathbf{u}$, with $\mathbf{u}$ detached from the computation graph to ensure correct partial derivatives.

Authors' Contact Information: JaeHyun Lee, University of Wisconsin-Madison, Madison, Wisconsin, USA; Eftychios Sifakis, University of Wisconsin-Madison, Madison, Wisconsin, USA and NVIDIA, Santa Clara, California, USA.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SA Conference Papers '26, Kuala Lumpur, Malaysia*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2842-6/2026/12
<https://doi.org/10.1145/3829340.3842319>

Algorithm 1 HomogenizeRHS $\mathbf{f} \leftarrow \text{Homogenize}(\mathbf{f}, \mathbf{g}, \dots)$

Input: RHS $\mathbf{f}$, Boundary values $\mathbf{g}$, Kernels $\mathbf{K}$, Masks $\mathbf{m}$

Output: Homogenized RHS $\hat{\mathbf{f}}$

► 1. Dirichlet Contribution

1: $\mathbf{u}_{\mathcal{D}} \leftarrow \mathbf{m}_{\mathcal{D}} \mathbf{g}$

2: $\mathbf{y}_{\mathcal{D}} \leftarrow \boldsymbol{\alpha} \text{Conv}(\mathbf{u}_{\mathcal{D}}, \mathbf{K}_{\nabla^2}) + \omega_{\text{eff}} \cdot \text{Conv}(\mathbf{u}_{\mathcal{D}}, \mathbf{K}_{\nabla}) + \boldsymbol{\sigma} \mathbf{u}_{\mathcal{D}}$

► 2. Natural Boundary Contribution

3: $\mathbf{c}_{\mathcal{N}} \leftarrow \sum_p \left(\boldsymbol{\alpha} \mathbf{w}_p^{(\nabla^2)} + \omega_{\text{eff}} \cdot \mathbf{w}_p^{(\nabla)} \right) \mathbf{m}_p \left(\frac{2h}{b} \mathbf{g} \right)$

4: **return** $\mathbf{f} - \mathbf{y}_{\mathcal{D}} - \mathbf{c}_{\mathcal{N}}$

► 3. Update RHS

Algorithm 2 ComputeDiagonal $\mathbf{D}$

Input: Parameters θ , Kernels $\mathbf{K}$, Masks $\mathbf{m}$

Output: Diagonal map $\mathbf{D}$

► 1. Uniform diagonal component

1: $\mathbf{d}_0 \leftarrow \boldsymbol{\alpha} \mathbf{w}_0^{(\nabla^2)} + \boldsymbol{\sigma}$

► 2. Boundary correction component

2: $\mathbf{d}_{\text{corr}} \leftarrow -\frac{2ha}{b} \sum_p \mathbf{m}_p \left(\boldsymbol{\alpha} \mathbf{w}_p^{(\nabla^2)} + \omega_{\text{eff}} \cdot \mathbf{w}_p^{(\nabla)} \right)$

3: $\mathbf{D} \leftarrow \mathbf{m}_{\text{comp}}(\mathbf{d}_0 + \mathbf{d}_{\text{corr}})$

► 3. Combine and Mask

4: **return** $\mathbf{D}$

Algorithm 3 Convolutional V-Cycle $\mathbf{u} \leftarrow \text{VCycle}(\mathbf{u}, \mathbf{f}, l)$

Input: Level l , Guess $\mathbf{u}$, RHS $\mathbf{f}$, Hierarchy $\{(\mathbf{K}, \mathbf{m}, \mathbf{D})_l\}$

Output: Improved solution $\mathbf{u}$

1: **if** $l = l_{\text{max}}$ **then**

► Base case: Coarsest solve (e.g., BiCGSTAB)

2: **return** KrylovSolve($\mathbf{u}, \mathbf{f}, \mathcal{H}_l$)

3: **end if**

► 1. Pre-Smoothing (Relaxation)

4: **for** $k = 1 \dots v_{\text{pre}}$ **do**

5: $\mathbf{r} \leftarrow \mathbf{f} - \text{ApplyOperator}(\mathbf{u}, \mathcal{H}_l)$

6: $\mathbf{u} \leftarrow \mathbf{u} + \omega \mathbf{D}_l^{-1} \mathbf{r}$

► Weighted Jacobi update

7: **end for**

► 2. Coarse Grid Correction

8: $\mathbf{r} \leftarrow \mathbf{f} - \text{ApplyOperator}(\mathbf{u}, \mathcal{H}_l)$

► Recompute residual

9: $\mathbf{f}_c \leftarrow \text{Conv}(\mathbf{r}, \mathbf{K}_R, \text{stride} = 2)$

► Downsample (Full weighting)

10: $\mathbf{e}_c \leftarrow \text{VCycle}(\mathbf{0}, \mathbf{f}_c, l + 1)$

► Solve for error on coarse grid

11: $\mathbf{u} \leftarrow \mathbf{u} + \text{ConvTranspose}(\mathbf{e}_c, \mathbf{K}_p, \text{stride} = 2)$

► Upsample correction

► 3. Post-Smoothing

12: **for** $k = 1 \dots v_{\text{post}}$ **do**

13: $\mathbf{r} \leftarrow \mathbf{f} - \text{ApplyOperator}(\mathbf{u}, \mathcal{H}_l)$

14: $\mathbf{u} \leftarrow \mathbf{u} + \omega \mathbf{D}_l^{-1} \mathbf{r}$

15: **end for**

16: **return** $\mathbf{u}$

Algorithm 4 ApplyAdjoint $\mathbf{v} \mapsto \mathbf{L}^\top \mathbf{v}$ **Input:** Adjoint vector $\mathbf{v}$, Kernels $\mathbf{K}$, Params θ , Masks $\mathbf{m}$ **Output:** Adjoint action $\mathbf{y}_{\text{adj}} = \mathbf{L}^\top \mathbf{v}$

```

  ▶ 1. Pre-multiply coefficients (Adjoint reverses order)
1:  $\mathbf{v}_\alpha \leftarrow \alpha \mathbf{v}$ ;  $\mathbf{v}_\omega \leftarrow \omega_{\text{eff}} \cdot \mathbf{v}$ ;  $\mathbf{v}_\sigma \leftarrow \sigma \mathbf{v}$ 
  ▶ 2. Transposed Uniform Convolution
2:  $\mathbf{y}_0 \leftarrow \text{Conv}(\mathbf{v}_\alpha, \mathbf{K}_{\nabla^2}) - \text{Conv}(\mathbf{v}_\omega, \mathbf{K}_\nabla) + \mathbf{v}_\sigma$ 
  ▶ 3. Transposed Boundary Correction
3:  $\mathbf{z}_p := \mathbf{m}_p \left( \mathbf{w}_p^{(\nabla^2)} \mathbf{v}_\alpha + \mathbf{w}_p^{(\nabla)} \cdot \mathbf{v}_\omega \right)$  ▶ Weighted masking
4:  $\mathbf{c}_{\text{adj}} \leftarrow \sum_p \left[ (\mathbf{S}_p * \mathbf{z}_p) - \frac{2ha}{b} \mathbf{z}_p \right]$  ▶ Sum of inverse fetches
5:  $\mathbf{y}_{\text{adj}} \leftarrow \mathbf{m}_{\text{comp}}(\mathbf{y}_0 + \mathbf{c}_{\text{adj}})$  ▶ 4. Combine
6: return  $\mathbf{y}_{\text{adj}}$ 

```

Algorithm 5 ComputeGradients $\frac{\partial \mathcal{L}}{\partial \mathbf{f}}, \frac{\partial \mathcal{L}}{\partial \theta} \leftarrow \text{Backward}(\mathbf{u}, \frac{\partial \mathcal{L}}{\partial \mathbf{u}}, \theta)$ **Input:** Solution $\mathbf{u}$, Loss grad $\mathbf{g}_u := \partial \mathcal{L} / \partial \mathbf{u}$, Params θ **Output:** Gradients $\mathbf{g}_f := \partial \mathcal{L} / \partial \mathbf{f}$, $\mathbf{g}_\theta := \partial \mathcal{L} / \partial \theta$

```

  ▶ 1. Adjoint Solve (Implicit Differentiation)
1:  $\lambda \leftarrow \mathbf{0}$  ▶ Initialize guess
2: Define  $\mathbf{L}^\top(\mathbf{v}) := \text{ApplyAdjoint}(\mathbf{v}, \dots)$  ▶ Using Alg. 4
3:  $\lambda \leftarrow \text{KrylovSolve}(\lambda, \mathbf{g}_u, \mathbf{L}^\top)$  ▶ Solve  $\mathbf{L}^\top \lambda = \mathbf{g}_u$ 
4:  $\mathbf{g}_f \leftarrow \lambda$ 
  ▶ 2. Parameter Gradient via VJP (PyTorch-style)
5:  $\mathbf{u}.\text{requires\_grad}(\text{False})$  ▶ Fix gradient: treat  $\mathbf{u}$  as constant
6:  $\mathbf{z} \leftarrow \text{ApplyOperator}(\mathbf{u}, \theta)$  ▶ Forward eval  $\mathbf{L}(\theta)\mathbf{u}$ 
7:  $\mathbf{z}.\text{backward}(\lambda)$  ▶ Autodiff VJP: accumulates into  $\theta.\text{grad}$ 
8:  $\mathbf{g}_\theta \leftarrow -\theta.\text{grad}$  ▶ Eq. 16:  $-\lambda^\top (\frac{\partial \mathbf{L}}{\partial \theta} \mathbf{u})$ 
9: return  $\mathbf{g}_f, \mathbf{g}_\theta$ 

```

2 Detailed Delineation from Closely Related Work

In this supplementary section, we provide a more detailed delineation from the two lines of work most closely related to ours in spirit: recent efforts to implement PDE solvers on ML stacks in computational physics, and PhiFlow [Holl et al. 2020] in computer graphics.

Computational-Physics PDE Solvers on ML Stacks. Recent work in computational physics, including work by Pain et al. [2024; 2026; 2025; 2025], has also explored mapping stencil computations to convolution calls in modern tensor libraries. These works aim to express a broad class of PDE solvers within AI-oriented software stacks, and are therefore closely related to our motivation. Related ML-stack PDE frameworks such as JAX-Fluids [Bezgin et al. 2023] are also relevant in spirit, but target rather different numerical settings, for example compressible flow solved with Riemann-solver-based methods rather than the elliptic subproblems emphasized here.

The main distinction lies in the target setting. In these works, boundary handling is typically much simpler, often assuming regular box domains where boundary conditions can be imposed through standard padding strategies. In such cases, the convolutional formulation remains relatively straightforward, since values outside the computational domain can be filled in a uniform manner (e.g., through zero-padding or other fixed padding rules). However, this strategy does not directly extend to the irregular boundary settings that are common in computer graphics. When the domain contains complex internal geometry, a single exterior node may be adjacent to multiple interior nodes with different local boundary relationships, so there is no unambiguous padding value that can be assigned independently of the target stencil evaluation.

For this reason, our focus is narrower in PDE class but substantially richer in geometric and boundary complexity. Rather than targeting broad PDE coverage, we focus on elliptic problems that are especially central in computer graphics, and we develop the operator lexicon needed to handle irregular boundaries, sparse domains, and inverse problems within that setting. In addition, our work goes beyond formulation alone by showing how this abstraction interfaces with sparse backends and by presenting a case study of targeted backend optimization.

PhiFlow. PhiFlow [Holl et al. 2020] is a high-level differentiable framework for fluid simulation in computer graphics and machine learning area, built on ML stacks such as PyTorch and JAX. It provides a convenient interface for standard operator-splitting pipelines, supports irregular geometries, and enables end-to-end differentiation. It is therefore one of the closest existing systems to our work from the graphics side. The main differences are:

- **Philosophy.** PhiFlow is designed as a high-level simulation framework, whereas our goal is to expose a reusable operator lexicon for constructing elliptic solvers. In particular, PhiFlow does not expose the convolutional/operator decomposition of solver components themselves, making backend optimization targets less explicit. Our dense benchmark suggests that this additional abstraction can introduce substantial overhead in 3D settings, whereas our more explicit operator interface makes the main computational kernels easier to identify and optimize.
- **General elliptic operators.** PhiFlow primarily targets solver settings compatible with symmetric positive definite systems. It does not provide the same general elliptic-operator formulation for nonsymmetric systems, nor the corresponding transpose operators needed for adjoint-based differentiation of such systems.
- **Sparse domains.** PhiFlow does not target sparse-domain execution as a first-class setting. Our formulation instead separates solver logic from the data representation, allowing the same operator pipeline to be realized on both dense tensors and sparse active-set backends.
- **Boundary treatment.** For irregular geometries, PhiFlow typically implements Laplacian evaluation through gradient computation, Neumann masking, and a subsequent divergence evaluation, effectively translating Neumann conditions on u into Dirichlet constraints on ∇u . While effective for

constructing Laplacian-like operators, this treatment is specific to that operator structure and does not directly provide reusable local boundary-correction primitives. It also does not naturally extend to more general boundary conditions such as Robin conditions.

- **Solver scope.** PhiFlow supports matrix-free solves through methods such as CG and BiCGSTAB, but additional matrix-free solvers must be implemented manually within its framework. In contrast, our operator formulation exposes the diagonal, transpose, and inter-grid transfer operators needed to build richer solvers, including multigrid and multigrid-preconditioned Krylov methods. As a concrete example, our supplemental material includes a 77-line PyTorch implementation of a full multigrid solver for a Poisson problem.

Bottleneck in PhiFlow. PhiFlow evaluates irregular-boundary Laplacians indirectly through a sequence of gradient evaluation, Neumann masking, and divergence evaluation. In our profiling, the main cost of this pipeline is not arithmetic, but memory traffic: the gradient, masking, and divergence stages create many intermediate tensors through padding, tensor reshaping, and temporary field construction. As a result, the computation becomes strongly memory-bound in 3D. By contrast, our formulation expresses the corresponding differential operations directly as convolutional stencil operators, reducing the need for this intermediate tensor pipeline. Even without dense-grid-specific kernel optimization, our PyTorch implementation is substantially faster than PhiFlow in the dense benchmark. Taichi remains faster on dense grids, but requires explicit loop and indexing code; our formulation keeps the solver in a higher-level operator language. As demonstrated in the sparse benchmark, the same low-channel stencil operators can also be targeted by backend specialization, suggesting a clear path to further dense-grid optimization.

3 Implementation Details

We provide detailed implementation specifications for the experiments and demonstrations presented in the main text and the accompanying supplementary video. A comprehensive summary of performance statistics, including grid resolution, backend frameworks, and execution times for each scene, is listed in Table S1.

3.1 Sparse Fluid Simulation

We implement three sparse MAC-grid examples using the active-region representation of fVDB: the decanter scene, the spherical benchmark, and the Trefoil scene. The decanter and Trefoil scenes run on a 1024^3 grid with $\Delta t = 0.0025$, while the spherical benchmark is evaluated at varying resolutions with $\Delta t = 0.01$. Since MAC quantities live at different staggered locations, their sparse topologies are not identical; we therefore build a union topology and use bit flags to indicate which quantities are active at each location.

Advection uses fVDB's `sample_trilinear` at backtraced positions. The decanter and spherical scenes use MacCormack advection [Selle et al. 2008]: after the correction step, we compute the min/max over the eight voxel corners bracketing the backtraced point using `ijk_to_index` gather operations. Invalid sparse corners are excluded from the reduction rather than treated as zero, and if no

valid corner exists we fall back to the uncorrected semi-Lagrangian value. The Trefoil scene uses RK3 backtracing without MacCormack correction.

Pressure projection computes divergence on cell centers, solves the pressure Poisson system, and subtracts the pressure gradient from face-centered velocities. The Trefoil scene uses conjugate gradients (CG), while the decanter and spherical benchmark use multigrid-preconditioned conjugate gradients (MGPCG). For MGPCG, we use a three-level multigrid hierarchy as the preconditioner, with weighted Jacobi smoothing using relaxation weight $\omega_J = 0.8$. For sparse differential operators, we use the optimized fVDB backend kernel described in the main paper: a single local-densification CUDA primitive realizes divergence, Laplacian, and gradient by changing only the stencil entry specification. We include a compact ~200-line sparse fluid implementation in the supplemental material.

3.2 Cooling Simulation

Time Discretization via Backward Euler. We model the cooling process using the transient convection-diffusion equation:

$$\frac{\partial u}{\partial t} + \mathbf{v} \cdot \nabla u = \nabla \cdot (\alpha \nabla u), \quad (\text{S1})$$

where u represents temperature, $\mathbf{v}$ is the velocity field, and α is the thermal diffusivity. Applying the Backward Euler method for time integration yields the implicit scheme:

$$\frac{u^{n+1} - u^n}{\Delta t} + \mathbf{v} \cdot \nabla u^{n+1} = \nabla \cdot (\alpha \nabla u^{n+1}). \quad (\text{S2})$$

Rearranging terms to fit our general linear operator form $\mathbf{L}u = \mathbf{f}$, where $\mathbf{L}$ includes diffusion, advection, and reaction terms, we obtain:

$$\nabla \cdot (\alpha \nabla u^{n+1}) - \mathbf{v} \cdot \nabla u^{n+1} - \frac{1}{\Delta t} u^{n+1} = -\frac{1}{\Delta t} u^n. \quad (\text{S3})$$

This allows us to solve for the next time step u^{n+1} by setting the effective reaction coefficient $\sigma = -1/\Delta t$, the advection field $\boldsymbol{\omega} = -\mathbf{v}$, and the source term $\mathbf{f} = -(1/\Delta t)u^n$.

Multigrid Configuration and Hyperparameters. We employ a geometric multigrid solver with a V-cycle scheme for both the fluid pressure projection and the heat diffusion solve. The hierarchy depth is set to $l_{\max} = 3$ levels, and BiCGSTAB is used as the coarsest level solver for robust convergence.

For the smoothing strategy, we use Red-Black Gauss-Seidel (RBGS) iterations with relaxation parameter $\omega = 1.0$ for both fluid and cooling simulations. However, we adjust the number of smoothing steps based on the conditioning of the PDE:

- **Fluid Simulation:** A single pre- and post-smoothing step ($\nu_{\text{pre}} = \nu_{\text{post}} = 1$) is sufficient for the pressure Poisson equation.
- **Cooling Simulation:** The convection-diffusion operator exhibits slower convergence properties compared to the pure Poisson operator. Therefore, we increase the smoothing steps to $\nu_{\text{pre}} = \nu_{\text{post}} = 2$ to ensure stability and convergence.

Physical Parameters. Both simulations use a fixed time step of $\Delta t = 0.015$. The thermal diffusivity α is spatially varying to distinguish materials:

- **Solid (David):** $\alpha_{\text{solid}} = 0.003$.

- **Fluid (Air):** $\alpha_{\text{fluid}} = 0.0015$.

To minimize numerical artifacts at the boundary, we define α at the solid-fluid interface as the arithmetic mean of the two values, i.e., $(\alpha_{\text{solid}} + \alpha_{\text{fluid}})/2$.

3.3 Coefficient Reconstruction

Ground Truth Configuration. For the synthetic ground truth setup, we define the conductivity coefficients as $\alpha_{\text{tumor}} = 3.0$ inside the tumor region and $\alpha_{\text{tissue}} = 0.1$ for the surrounding healthy tissue. Similar to the forward simulations, the coefficients at the interface are set to the arithmetic mean, $(\alpha_{\text{tumor}} + \alpha_{\text{tissue}})/2$, to represent a smooth transition.

Optimization Strategy. We adopt a Physics-Informed Neural Network (PINN)-style [Cai et al. 2021] optimization approach. Unlike the forward simulation, this inverse problem does not require inverting the matrix L . Instead, we directly minimize the PDE residual norm $\|L(\alpha)\mathbf{u} - \mathbf{f}\|_2^2$. Since the loss depends only on the forward evaluation of the operator, no linear solver (e.g., Multigrid or BiCGSTAB) is necessary. The gradients with respect to α are computed by back-propagating through a single call to `ApplyOperator` using standard automatic differentiation. We initialize α with Gaussian random noise and perform the optimization using the Adam optimizer with a learning rate of 0.01; we observe that the coefficient field is successfully reconstructed within 300 iterations.

3.4 Inverse Fluid Control

Problem setup. We optimize the initial MAC velocity field of a 2D smoke simulation so that the density at a prescribed target frame matches a SIGGRAPH logo image (frame 30). The initial density is a solid disk near the domain center, with its total mass matched to the target. The simulation uses a closed box with homogeneous Neumann pressure boundaries, and the normal MAC velocity components are masked to zero at solid walls. Each time step self-advects the velocity, performs pressure projection, and then advects the density with the projected velocity.

Optimization and adjoint solve. The optimization variable is the initial velocity field, updated with Adam. We minimize the mismatch between the simulated density at the target frame and the logo target, optionally with velocity regularization. To differentiate through pressure projection, we use an adjoint pressure solve: the forward

CG solve is executed without unrolling, and the backward pass solves the transposed linear system to propagate gradients through the implicit projection. Since the Neumann Laplacian in this example is self-adjoint, the transpose operator coincides with the forward operator, but we still use the same transpose-solve interface as in our general adjoint formulation.

References

- Deniz A Bezgin, Aaron B Buhendwa, and Nikolaus A Adams. 2023. JAX-Fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows. *Computer Physics Communications* 282 (2023), 108527.
- Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, and George Em Karniadakis. 2021. Physics-informed neural networks (PINNs) for fluid mechanics: A review. *Acta Mechanica Sinica* 37, 12 (2021), 1727–1738.
- Boyang Chen, Claire E Heaney, Jefferson LMA Gomes, Omar K Matar, and Christopher C Pain. 2024. Solving the discretised multiphase flow equations with interface capturing on structured grids using machine learning libraries. *Computer Methods in Applied Mechanics and Engineering* 426 (2024), 116974.
- Boyang Chen, Claire E Heaney, and Christopher C Pain. 2026. Neural Physics: Using AI Libraries to Develop Physics-Based Solvers for Incompressible Computational Fluid Dynamics. *Computers & Fluids* (2026), 106981.
- Boyang Chen, Amin Nadimy, Claire E Heaney, Mohammad Kazem Sharifian, Lluís Via Estrem, Ludovico Nicotina, Arno Hilberts, and Christopher C Pain. 2025. Solving the discretised shallow water equations using neural networks. *Advances in Water Resources* 197 (2025), 104903.
- Philipp Holl, Vladlen Koltun, Kiwon Um, and Nils Thuerey. 2020. phiflow: A differentiable pde solving framework for deep learning via physical simulations. In *NeurIPS workshop*, Vol. 2.
- Linfeng Li, Jiansheng Xiang, Boyang Chen, Claire E Heaney, Steven Dargaville, and Christopher C Pain. 2025. Implementing the discontinuous-Galerkin finite element method using graph neural networks with application to diffusion equations. *Neural Networks* 185 (2025), 107061.
- Andrew Selle, Ronald Fedkiw, Byungmoon Kim, Yingjie Liu, and Jarek Rossignac. 2008. An unconditionally stable MacCormack method. *Journal of Scientific Computing* 35, 2 (2008), 350–371.

Table S1. Performance statistics. For sparse simulations, the number in parentheses in the resolution column indicates the active voxel count. For fluid simulations, total time reports the full single simulation step, with linear solver time shown in parentheses. For inverse design, total time reports the full optimization process.

Scene	Backend	Resolution	Scheme	Iters	Total Time (ms)
Decanter (Sparse Fluid)	fVDB/CUDA	1024 ³ (80.6M)	MGPCG	5	7,767 (5,650)
Sphere (Sparse Fluid)	fVDB/CUDA	512 ³ (70.3M)	MGPCG	5	6,115 (4,963)
Trefoil (Sparse Fluid)	fVDB/CUDA	1024 ³ (1.26M)	CG	100	222 (157)
David (Fluid)	PyTorch/CUDA	257 ³	MG	5	551 (536)
David (Cooling)	PyTorch/CUDA	257 ³	MG	5	1,352 (943)
Tumor Recon (Inverse)	PyTorch/CUDA	128 ³	Autodiff (Adam)	300	3,816.70
Fluid Control (Inverse)	PyTorch/CUDA	256 ²	Adjoint CG (Adam)	30	69,364